# SIMATS ENGINEERING

## CO3 - ASSESSMENT TOOL 3

## Comparative Analysis

**COURSE NAME: DEEP LEARNING** **COURSE CODE: MLA0408**

---

**COURSE OUTCOME COVERED**

**CO3:** Students will be able to understand, analyze and apply CNN concepts including layers, filters, parameter sharing, regularization, AlexNet and ResNet for real-world image processing applications. (BTL-6)

CO Weightage: 25%

Assessment Tool Weightage: 40%

**QUESTIONS:** **Total Marks:25**

Q1. CNN vs Fully Connected Neural Network

A medical imaging organization wants to classify X-ray images using deep learning. Compare a **CNN with a traditional fully connected neural network** in terms of architectural design, motivation, spatial feature extraction, number of parameters, computational complexity, and ability to handle image data. Analyze the roles of convolutional layers, filters, and pooling layers, and justify why CNNs are more suitable for image classification applications.

Q2. Different CNN Layers and Filters

A manufacturing company is developing a CNN-based system to detect defects in industrial products. Compare the functions of **convolutional layers, pooling layers, activation layers, and fully connected layers** in terms of feature extraction, spatial information, computational requirements, and classification. Further compare **early-layer filters and deeper-layer filters** based on the type of features they learn, and justify how these layers collectively improve defect detection.

Q3. Parameter Sharing vs Independent Weights

A smart surveillance system needs to process thousands of high-resolution images efficiently. Compare **parameter sharing in CNNs with independent weights in fully connected networks** in terms of the number of parameters, memory requirements, computational complexity, feature detection, and translation-related invariance. Analyze how the reuse of convolutional filter weights improves learning efficiency and justify its importance for large-scale image-processing applications.

Q4. AlexNet vs ResNet

A computer vision company is developing an image classification system and is considering **AlexNet and ResNet** as possible architectures. Compare the two architectures in terms of network depth, layer organization, convolutional filters, parameter requirements, feature extraction capability, training difficulty, vanishing-gradient problem, computational cost, and classification performance. Recommend the more suitable architecture for a complex image recognition application and justify your choice.

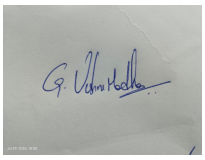
Q5. Regularized CNN vs Non-Regularized CNN

An agricultural organization is developing a CNN to classify plant diseases from leaf images. The initial model achieves very high training accuracy but performs poorly on unseen images. Compare a **CNN without regularization and a CNN using dropout, data augmentation, and batch normalization** in terms of overfitting, generalization, training stability, computational requirements, and model performance. Analyze which regularization techniques should be applied and justify the most appropriate approach for reliable real-world disease classification.

**Code of Conduct:**

I G Vishnu Madhav (192425192) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary actio

**Signature of the student:**



**RUBRICS FOR CALCULATION:**

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Selection of Studies/Parameters | 20% | Selects highly relevant and diverse studies with well-defined comparison parameters | Selects relevant studies with minor gaps in parameter definition | Limited or partially relevant selection | Poor or inappropriate selection of studies |
| Comparison & Differentiation | 25% | Clearly compares multiple aspects (methods, results, limitations) with strong differentiation | Good comparison with minor gaps | Basic comparison with limited depth | No clear comparison; mostly descriptive |
| Critical Evaluation | 25% | Critically evaluates strengths, weaknesses, and implications with strong justification | Provides evaluation with some justification | Limited evaluation; lacks depth | No critical evaluation provided |
| Synthesis & Insight Generation | 20% | Generates meaningful insights and conclusions from comparisons | Some insights derived from comparison | Limited or unclear insights | No insights generated |
| Clarity | 10% | Information is clearly presented (tables/figures) with logical structure | Generally clear with minor issues | Presentation lacks clarity or structure | Poor presentation and difficult to understand |

# Q1. CNN vs Fully Connected Neural Network

A **Convolutional Neural Network (CNN)** is specifically designed to process grid-like data such as images, whereas a **traditional Fully Connected Neural Network (FCNN)** treats every input feature independently and connects it to every neuron in the next layer.

| Aspect | CNN | Fully Connected Neural Network |
|---|---|---|
| Architecture | Convolution → Activation → Pooling → Fully connected/output | Fully connected layers throughout |
| Motivation | Designed to exploit spatial structure in images | Designed for general numerical/tabular features |
| Spatial feature extraction | Preserves and analyzes local spatial relationships | Spatial relationships are largely lost after flattening |
| Parameters | Relatively fewer because of parameter sharing | Very large number of parameters |
| Computational complexity | Lower for image processing | Very high for high-resolution images |
| Image handling | Naturally handles 2D/3D image structure | Usually converts image into a long 1D vector |
| Translation handling | Detects features even when their position changes | Less effective because each pixel has separate connections |
| Feature learning | Learns hierarchical features automatically | Learns features but without explicit spatial structure |

## Role of convolutional layers

The **convolutional layer** applies small filters, such as 3×3 or 5×5, across the image. Instead of connecting every pixel to every neuron, it examines local regions.

For example:

**Input X-ray → Convolution → Feature Map**

A filter may learn to detect:

- Edges
- Lines
- Boundaries

- Textures
- Shapes

As the network becomes deeper, the learned features become more complex.

## Role of filters

Filters are small matrices of learnable weights.

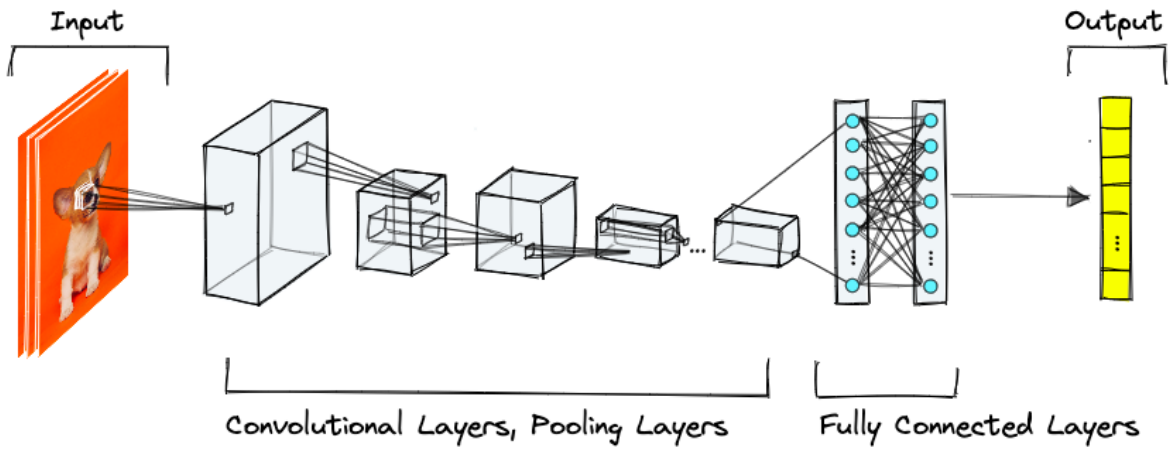
For example, an early filter might detect an edge:

[ -1  -1  -1 ]

[ 0  0  0 ]

[ 1  1  1 ]

During training, the CNN automatically adjusts these weights to detect useful patterns in X-ray images.




## Role of pooling

Pooling reduces the spatial dimensions of feature maps.

Common types:

- **Max pooling** – selects the maximum value.
- **Average pooling** – calculates the average value.

Pooling:

- Reduces computation
- Reduces memory requirements
- Helps retain important features

### Why CNN is better for X-ray classification

An X-ray contains important spatial relationships—for example, the location and shape of abnormal regions matter. CNNs preserve these relationships while learning features automatically.

A fully connected network would require an enormous number of weights when processing a high-resolution X-ray.

# Q2. Different CNN Layers and Filters

A CNN for industrial defect detection generally consists of:

**Input → Convolution → Activation → Pooling → Fully Connected → Output**

### Comparison of CNN layers

| Layer | Main function | Feature extraction | Spatial information | Computation |
|---|---|---|---|---|
| Convolution | Applies filters to input | Extracts local features | Preserves spatial arrangement | Moderate |
| Activation | Introduces non-linearity | Helps learn complex patterns | Preserves feature-map structure | Low |
| Pooling | Reduces dimensions | Retains important features | Reduces spatial resolution | Low |
| Fully Connected | Performs final classification | Combines learned features | Spatial structure mostly removed | High |

### 1. Convolutional layer

The convolutional layer scans the product image using filters.

It can detect:

- Cracks
- Scratches
- Holes
- Broken edges
- Surface irregularities
- Texture variations

The output is called a **feature map**.

### 2. Activation layer

A common activation function is **ReLU**:

**ReLU(x) = max(0, x)**

It converts negative values to zero and allows the network to learn non-linear relationships.

Without activation functions, stacking multiple convolutional layers would behave approximately like a single linear transformation.

### 3. Pooling layer

Pooling decreases the size of feature maps.

For example:

4 × 4 feature map

↓

2 × 2 Max Pooling

↓

2 × 2 feature map

This reduces computation while retaining strong responses corresponding to important defects.

### 4. Fully connected layer

The final learned features are combined by fully connected layers.

The network can then classify the product as:

**Normal / Defective**

or:

**Scratch / Crack / Dent / Hole / Normal**

# Q3. Parameter Sharing vs Independent Weights

## Basic idea

In a fully connected network, different connections generally have different weights.

In a CNN, the same convolutional filter weights are reused across different locations of an image. This is called parameter sharing.

## Comparison

| Aspect | CNN with Parameter Sharing | Fully Connected Network |
|---|---|---|
| Weights | Same filter reused spatially | Separate weights for connections |
| Number of parameters | Much smaller | Very large |
| Memory | Lower | Higher |
| Computation | More efficient for images | Expensive for high-resolution images |
| Feature detection | Same feature detected anywhere | Location-specific learning |
| Translation handling | Better | Poorer |
| Learning efficiency | High | Lower for image patterns |
| Scalability | Suitable for large images/datasets | Difficult for high-resolution images |

## Example

**Suppose an image has 1,000 × 1,000 pixels.**

**That gives:**

1,000,000 input values

If a fully connected layer has 1,000 neurons:

Parameters ≈ 1,000,000 × 1,000 = 1 billion weights

**This creates huge memory and computational requirements.**

**Now consider a CNN with a 3 × 3 filter.**

**Only:**

3 × 3 = 9 weights

are required for one filter, plus a bias.

Those same 9 weights can be applied throughout the image.

Why parameter sharing is important

Suppose a CNN learns a filter for detecting a particular edge.

That filter can detect the same edge:

- At the top of the image
- At the bottom
- On the left
- On the right
- At different locations

**Therefore, the CNN does not need to learn a separate edge detector for every pixel location.**

## Translation-related feature detection

Parameter sharing makes CNNs equivariant to translations at the convolution stage: when an object moves, the corresponding feature response also moves.

Pooling and other architectural choices can then provide some degree of translation invariance, making classification less sensitive to small changes in object location.

## Conclusion

**Parameter sharing dramatically reduces:**

- Number of parameters
- Memory usage
- Training requirements
- Computational cost

Therefore, it is essential for processing thousands of high-resolution surveillance images efficiently.

# Early-layer filters vs deeper-layer filters

| Early layers | Deeper layers |
|---|---|
| Detect simple features | Detect complex features |
| Edges | Shapes |
| Lines | Object parts |
| Corners | Defect patterns |
| Simple textures | Complete defect structures |
| Small receptive fields | Larger effective receptive fields |

**For example:**

Layer 1: detects edges
↓
Layer 2: detects curves and textures
↓
Layer 3: detects shapes and surface patterns
↓
Deep layers: recognize a complete crack or defect

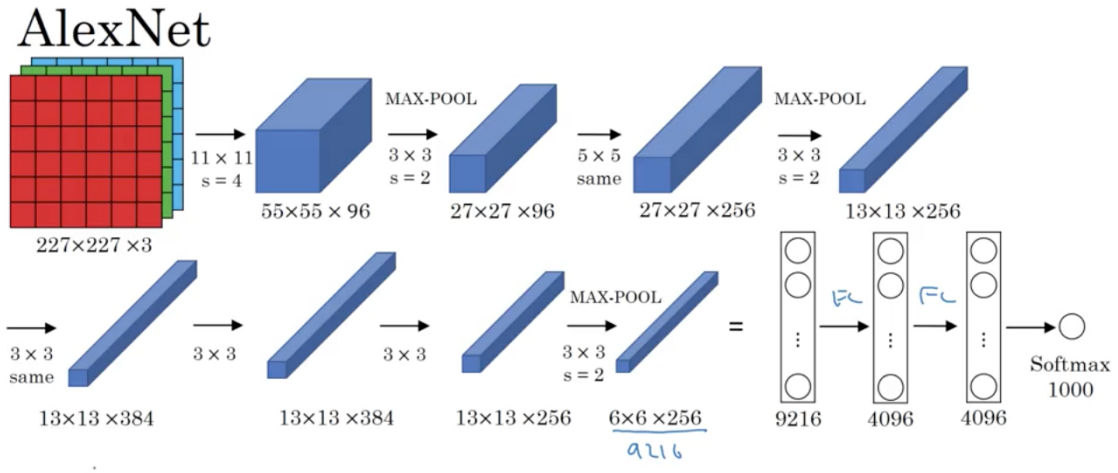
## How the layers improve defect detection

The layers work hierarchically:

Raw image → edges → textures → shapes → defect patterns → defect class

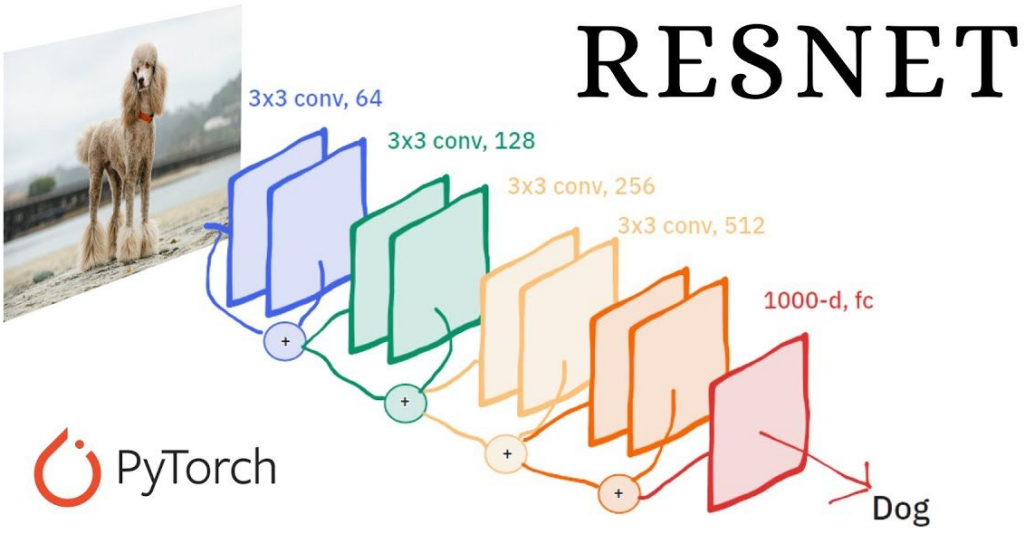
Thus, CNN layers allow the system to detect both simple and complex defects automatically.

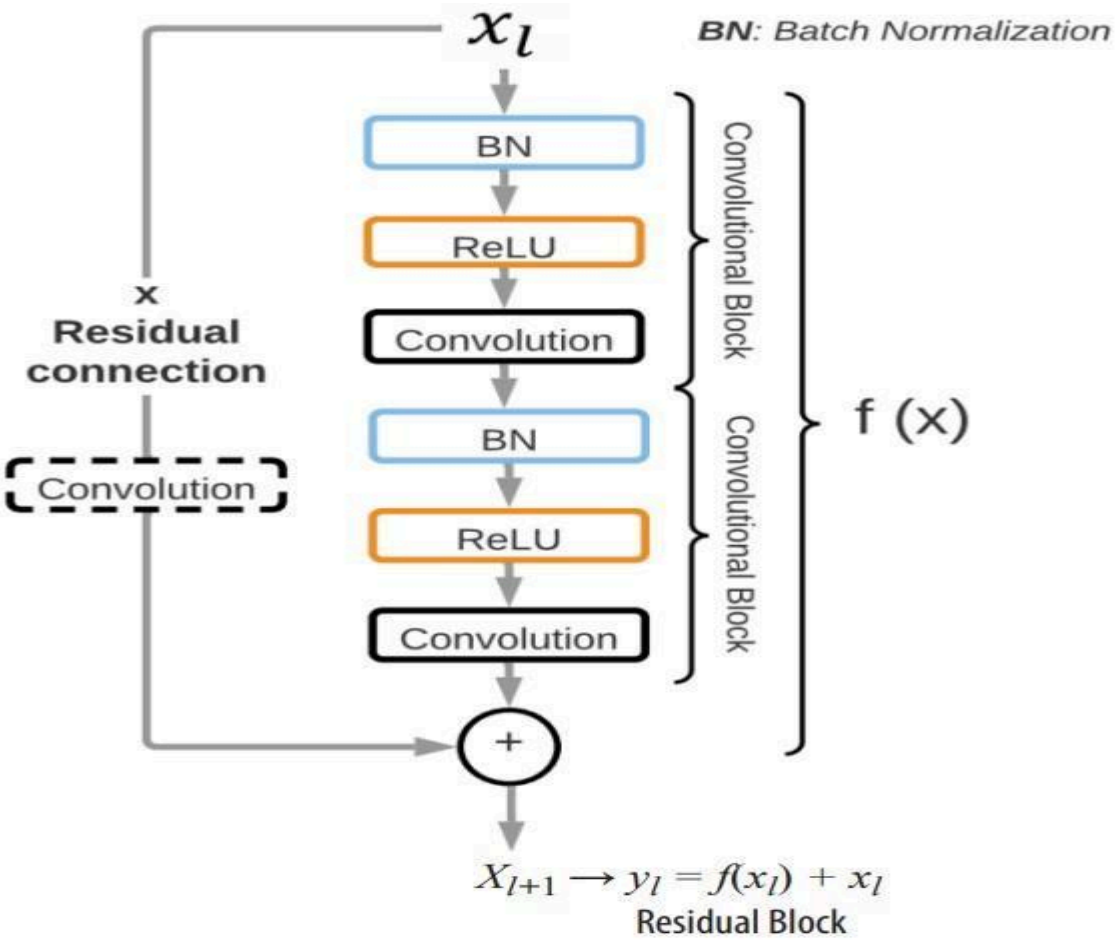
# Q4. AlexNet vs ResNet

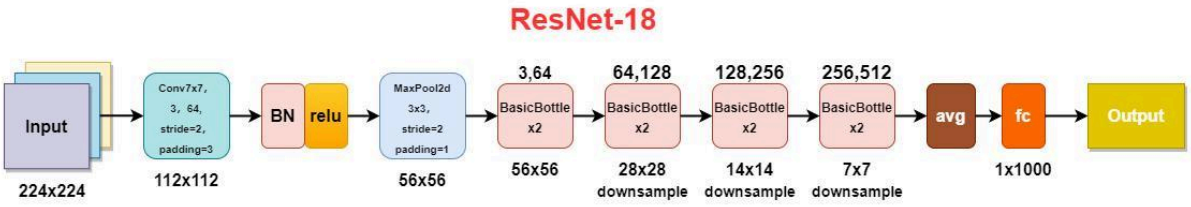


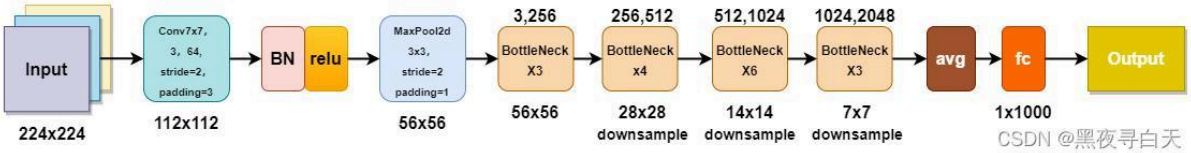

[Krizhevsky et al., 2012. ImageNet classification with deep convolutional neural networks]

Andrew Ng






5

**AlexNet** and **ResNet** are important CNN architectures, but ResNet is substantially deeper and uses residual connections to make deep networks easier to train.

## ComparisonAlexNet architecture

| Aspect | AlexNet | ResNet |
|---|---|---|
| **Introduced** | 2012 | 2015 |
| **Depth** | Relatively shallow | Can be very deep, e.g., ResNet-50/101/152 |
| **Basic organization** | Convolution + pooling + fully connected | Convolution + residual blocks |
| **Filters** | Uses convolutional filters of different sizes | Primarily uses smaller convolutional filters in residual blocks |
| **Parameters** | About 60 million | Depends on version; ResNet-50 has about 25 million |
| **Feature extraction** | Good hierarchical features | More powerful deep hierarchical features |
| **Training difficulty** | Easier because it is shallower | Deep networks are difficult without residual connections |
| **Vanishing gradient** | Less severe due to shallower depth | Residual connections greatly reduce the problem |
| **Computational cost** | Lower than very deep ResNets | Higher for deep variants |
| **Performance** | Strong for its time | Generally much better on complex modern recognition tasks |

A simplified structure is:

**Input image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Fully Connected → Output**

AlexNet was historically important because it demonstrated the effectiveness of deep CNNs for large-scale image recognition.

It introduced important techniques such as:

- ReLU activation
- Dropout
- GPU-based training
- Data augmentation

## ResNet architecture

ResNet introduced the **residual block**.

Instead of forcing a layer to learn a complete transformation:

**Output = F(x)**

it learns a residual:

**Output = F(x) + x**

The original input **x** is passed through a shortcut connection and added to the learned transformation.

# Q5. Regularized CNN vs Non-Regularized CNN

The agricultural CNN has **very high training accuracy but poor performance on unseen leaf images**. This is a classic indication of **overfitting**.

### Comparison

| Aspect | CNN without regularization | CNN with Dropout + Data Augmentation + Batch Normalization |
|---|---|---|
| **Overfitting** | High | Reduced |
| **Generalization** | Poor | Better |
| **Training stability** | Can be unstable | Generally more stable |
| **Training data requirement** | More sensitive to limited data | Data augmentation effectively increases variety |
| **Memory/computation** | Lower during training | Some additional computation |
| **Performance on unseen images** | Often poor | Usually improved |
| **Robustness** | Lower | Higher |

## 1. Dropout

Dropout randomly disables some neurons during training.

For example:

Before dropout:

● ● ● ● ● ●

During training:

● ○ ● ○ ● ●

The network cannot depend heavily on a small group of neurons.

**Benefits**

- Reduces overfitting
- Encourages distributed feature learning
- Improves generalization

Dropout is particularly useful in the **fully connected layers**.